

Laya as a Tool-Call Router for an LLM - Benchmark Report

What this is

Experiments on whether laya (a small encoder + decision head, NOT a generative LLM) can decide which external tools an LLM should call for a given user message. laya answers typed questions about a 'state' over HTTP (load_laya.py, /evaluate) in ~0.2-1s per question batch on CPU, returning calibrated probabilities. The LLM is never queried here - we only benchmark the routing decision.

Question types used

choice : pick exactly one label from named options -> {choice, probabilities, confidence}
noul : yes/no -> P(yes) in 0..1. Used once per tool for multi-label decisions.
score : rate on an ordered scale -> expected level. Tried as a 'how many tools' counter (failed).

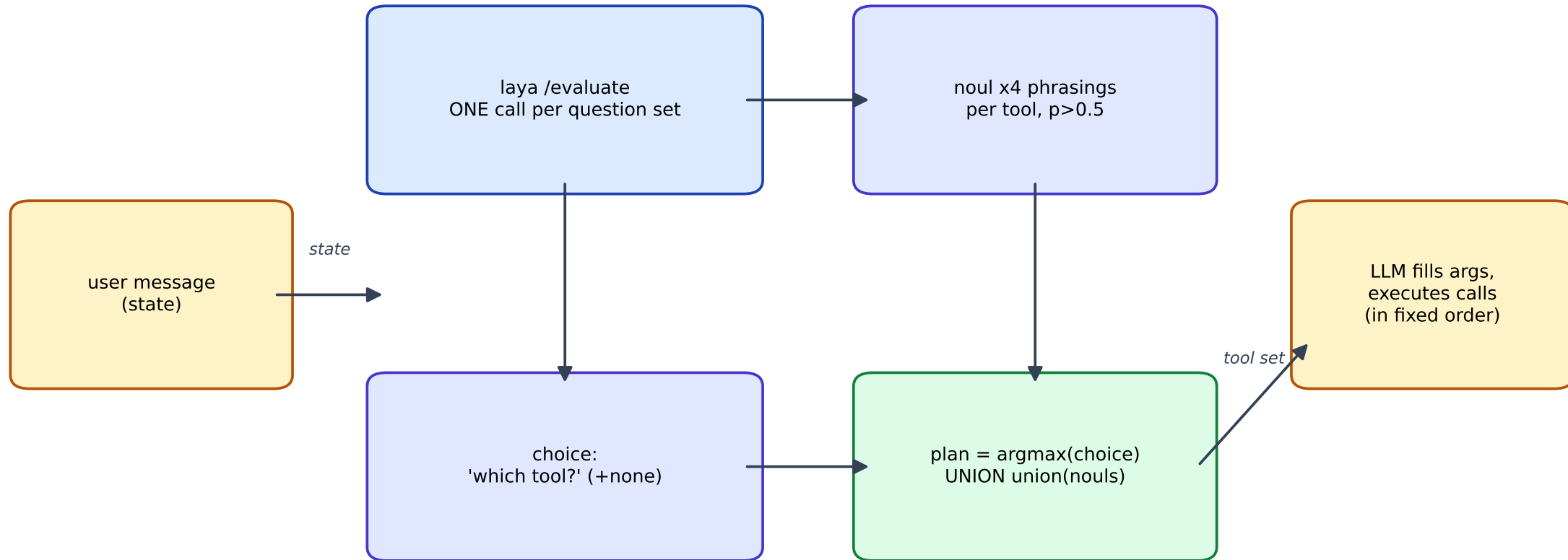
Setup

Server : load_laya.py on 127.0.0.1:8000, english checkpoint (512 ctx), device=cpu
Tools : get_weather, web_search, calculator, send_email (+ 'none' in the choice variant)
Data : 12-14 hand-labeled user messages per experiment - single-tool, multi-tool, no-tool

Headline result

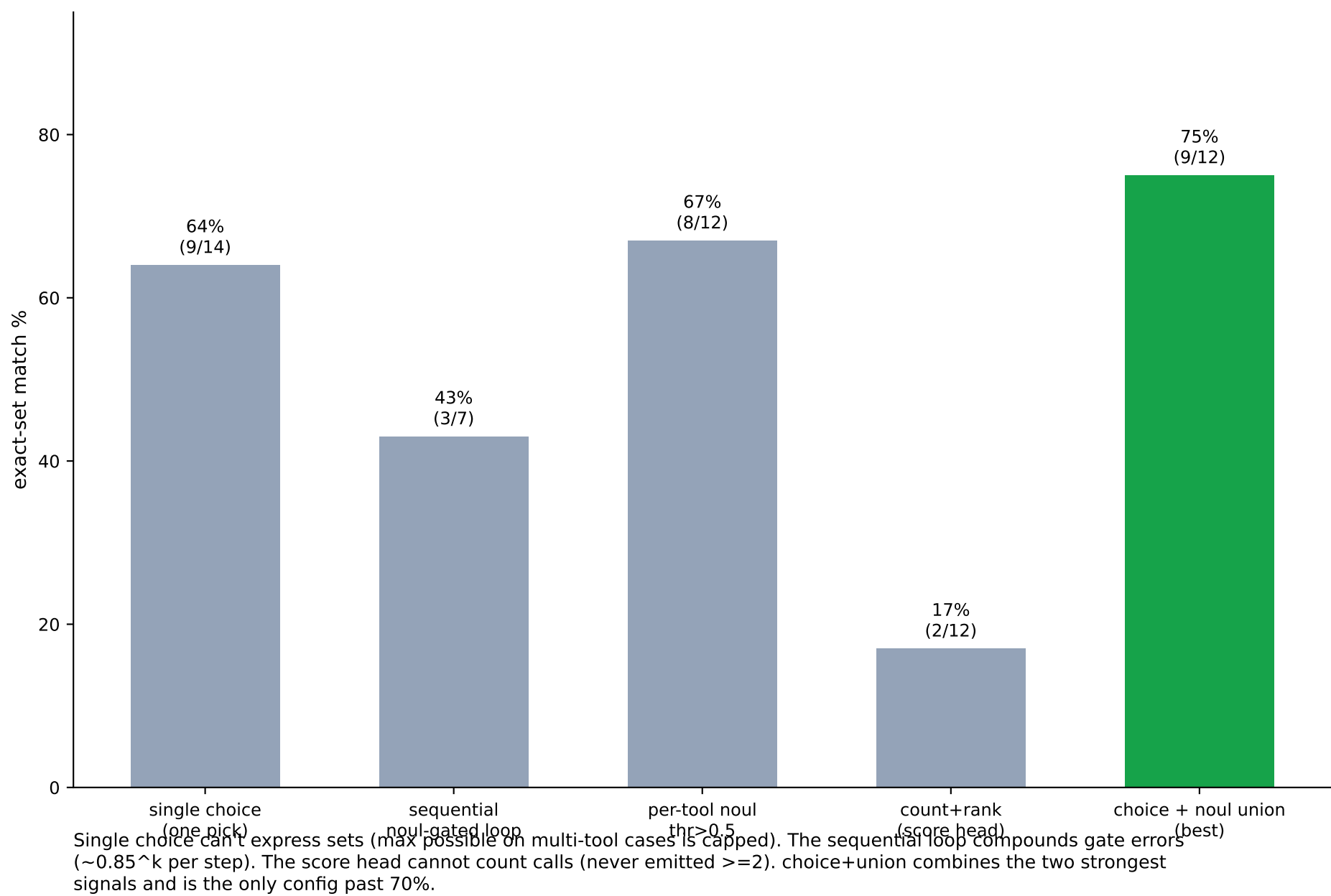
Best algorithm: take the single 'choice' argmax (unless 'none') and UNION it with every tool whose per-tool noul exceeds 0.5 under ANY of 4 phrasings. Exact-set match 9/12 (75%), precision 91%, recall 77%. The choice head nails primary intent (100% precision alone); the noul union recovers secondary tools that the choice head cannot express.

Recommended pipeline (winning algorithm)

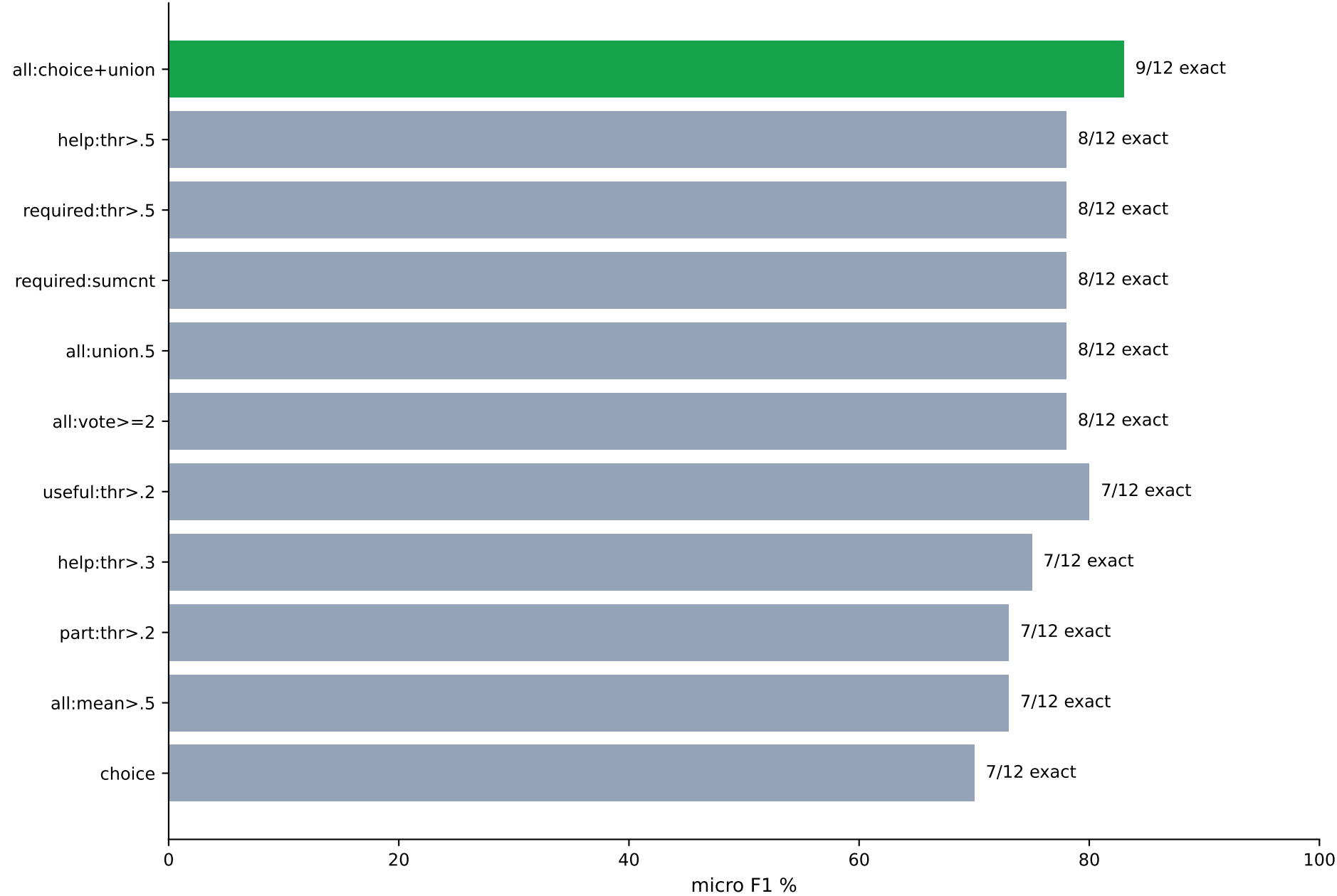


Cost: 5 /evaluate calls per message (1 choice + 4 noul batches), ~1s total on CPU, no LLM tokens spent on routing. Ordering is hardcoded by dependency (gather -> compute -> send); laya only decides the SET.

Approach comparison - exact-set match

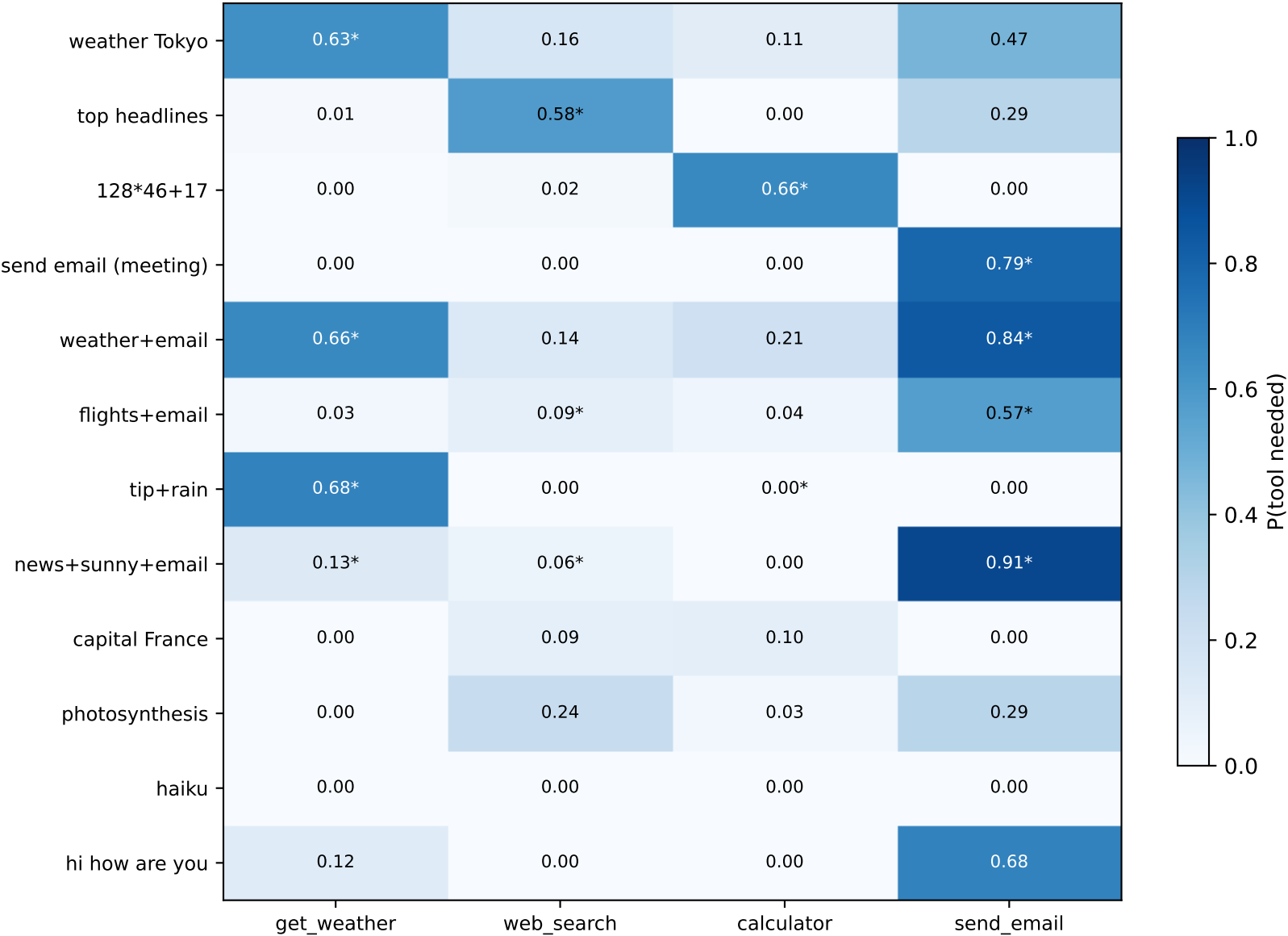


Algorithm sweep - top configs (36 evaluated)



Phrasing variants: help / part / required / useful. Rules: thresholds, relative-to-max, largest-gap split, expected-count, union/vote/mean across ph

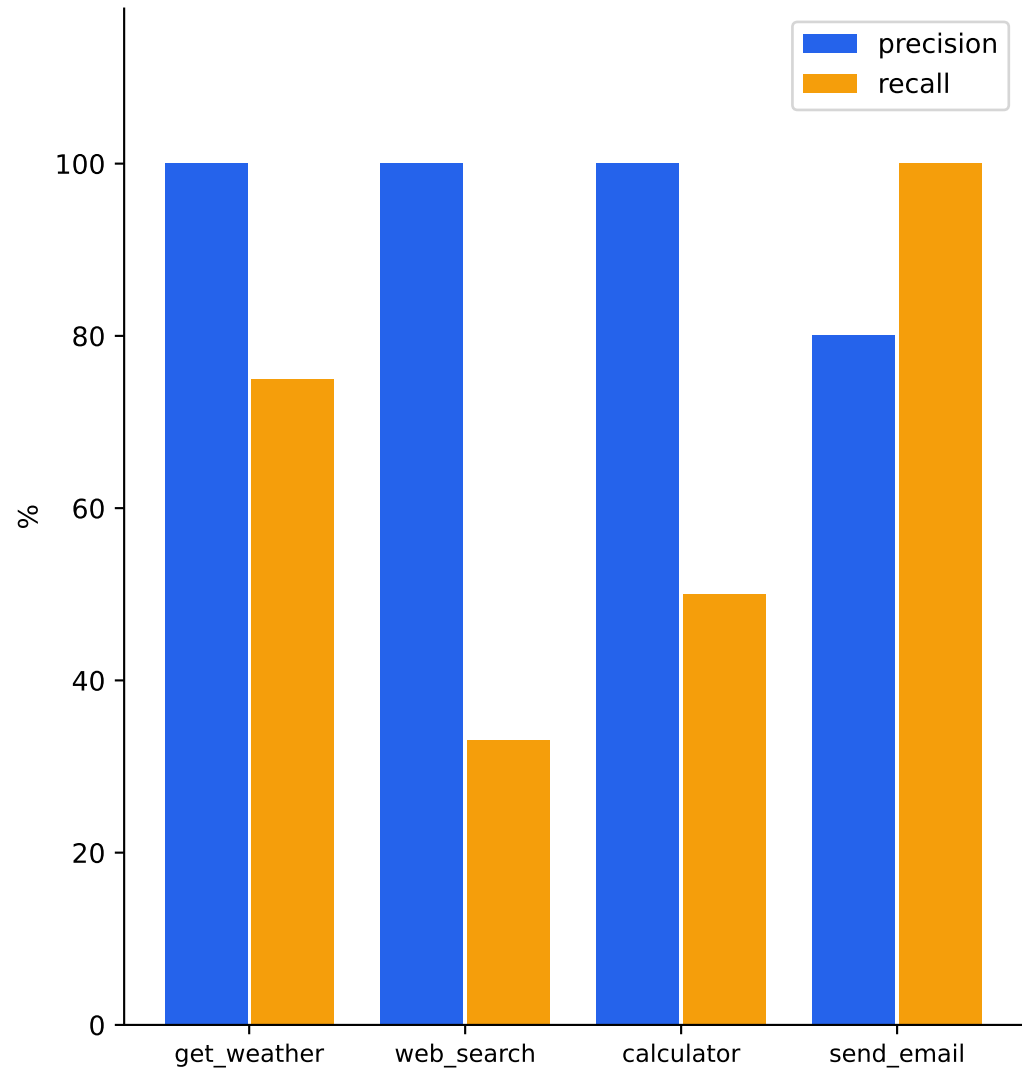
Per-tool noul probabilities per case ('help' phrasing)



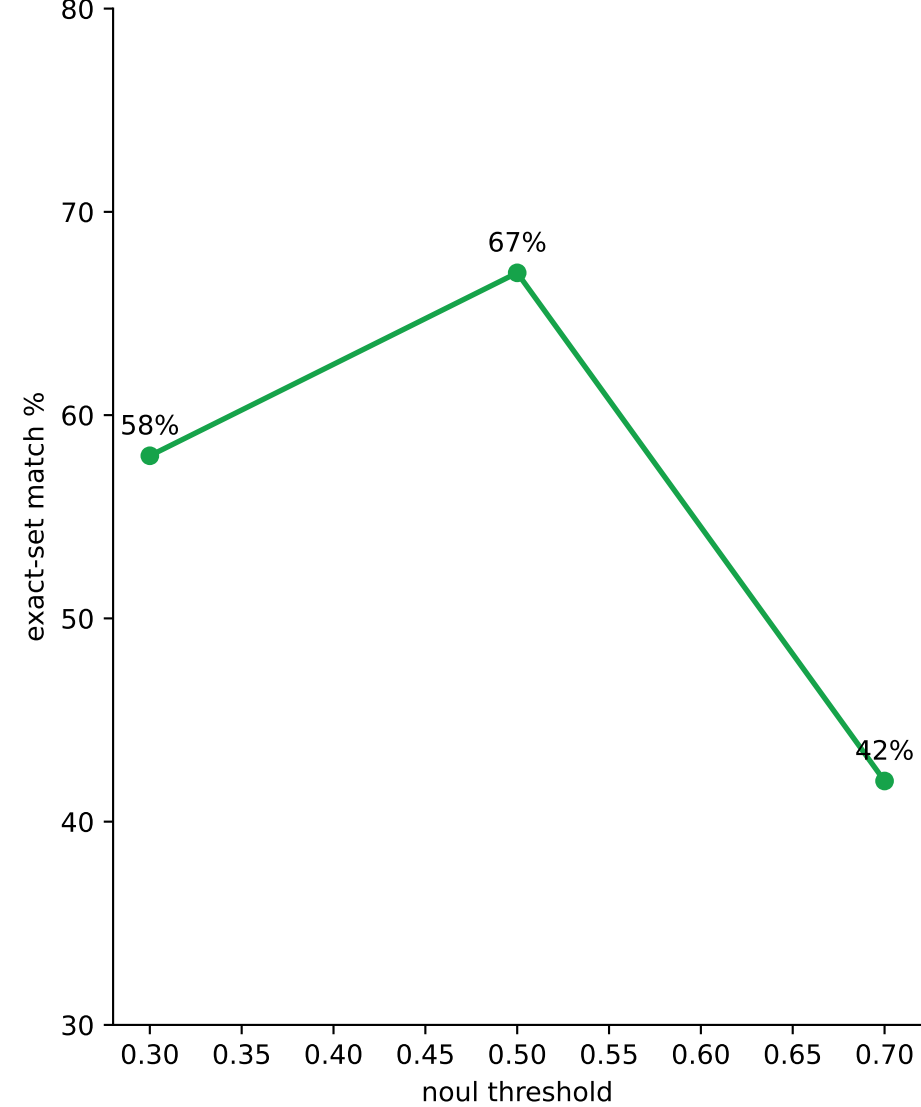
* = ground truth. The failure mode is visible: on multi-tool rows (4-7) the dominant intent absorbs the probability mass and secondary tools sit at <=0.14 - below any usable threshold. Row 11 ('hi how are you') shows the send_email false positive that fires under every phrasing.

Per-tool quality + threshold sensitivity (thr rule)

precision vs recall per tool (thr>0.5)

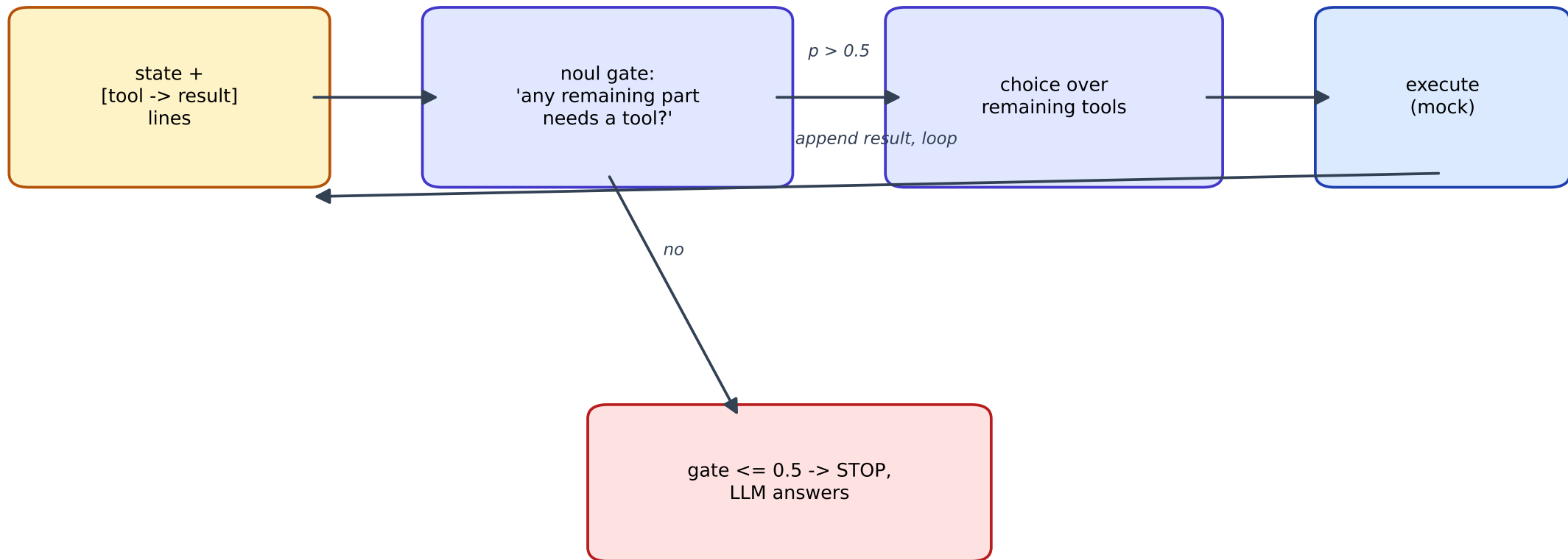


threshold sweep



Precision is uniformly high - laya almost never calls a wrong tool. Recall is the bottleneck, and only on secondary intents. The threshold optimum is 0.5: lowering to 0.3 gains nothing (the misses are near-zero, not borderline), raising to 0.7 loses real calls.

Rejected approach: sequential noul-gated loop



Scored 3/7 (43%). Each gate decision is ~80-85% reliable, so a k-call plan needs k+1 consecutive correct gates ($\sim 0.85^4 = 52\%$ for 3 tools). Also observed: send_email chosen BEFORE the data it should send existed, gates firing after completion, and gates not firing with work remaining. The loop amplifies the model's weaknesses instead of its strengths.

Findings, failure modes, recommendations

What works

Single-intent routing is reliable: choice alone = 100% precision, and per-tool noul's are $\geq 90\%$ precise. The best multi-tool config (choice + noul union, 9/12 / F1 83%) needs one /evaluate batch per phrasing and no re-asking.

What doesn't

The score head cannot count ('how many tool calls' stayed in 0.3-1.2 regardless of truth; count+rank = 17%). Sequential gating compounds errors (43%). Thresholds below 0.5 can't recover misses because secondary-tool probabilities are near-zero, not borderline.

Persistent failure modes

1. Secondary-intent suppression: in compound requests the dominant tool absorbs the mass ('flights + email' -> web_search ≤ 0.09 under all 4 phrasings).
2. send_email over-fires on casual chat ('Hi! How are you today?' -> 0.68-0.89 everywhere).
3. Ordering is not modeled: when the loop did fire, send_email was chosen before its inputs existed.
4. Instruction sensitivity: noul's must name the concrete tools (0.15 -> 0.72 when named).

Recommendations

- Production rule: plan = $\text{argmax}(\text{choice}) - \{\text{none}\} \cup \{t: \text{any noul phrasing} > 0.5\}$.
- Execute the plan in fixed dependency order (gather -> compute -> send); don't ask laya to order.
- Dict/JSON states degrade answers vs plain strings (0.72 -> 0.29 on the same message) - keep state a plain string.
- ~75% exact-set is likely near this checkpoint's ceiling on multi-intent inputs; pushing recall further needs training data with compound requests, not algorithm changes.
- Next step: wire the LLM/LFM in for argument filling; laya only produces the tool set.